2f. Tuples

Martin Alfaro
PhD in Economics

INTRODUCTION

We continue our exploration of *collections*, defined as objects storing multiple elements. Having previously focused on arrays, we'll now turn our attention to another form of collection known as tuples.

Tuples differ from arrays in several respects. First, they can comprise elements with heterogeneous types, without incurring any performance penalty. Instead, as we'll see in Part II of the book, arrays mixing disparate types (such as integers and strings) perform poorly. Another key distinction is that tuples have a **fixed size** and are **immutable**. This implies that, once a tuple is created, its elements can't be added, removed, or modified. Finally, tuples are only suitable for **storing a small number of elements**. In contrast, large tuples will result in slow computations at best, or directly trigger fatal errors. For this reason, vectors remain the appropriate choice for large collections.

At this stage, we'll limit our discussion to the basic features of tuples, introducing only what's necessary to develop the fundamentals of Julia. A more in-depth exploration will follow in Part II, after we've developed the necessary tools to appreciate their role in high-performance applications.

DEFINITION OF TUPLES

Tuples are defined by enclosing their elements within parentheses `()`, unlike the square brackets `[]` used in vectors. When a tuple contains more than one element, the parentheses are optional and often omitted.

Single-element tuples have stricter syntax rules, requiring parentheses `()` and a trailing comma `,` after the element. For example, a tuple with the single element `10` is represented as `(10,)`. The notation distinguishes a tuple from the expression `(10)`, which would simply be interpreted as the number `10`.

As with vectors, `x[i]` is the syntax for accessing the *i*-th element of a tuple `x`.

TUPLE

```
x = (4,5,6)
x =  4,5,6           #alternative notation
```

```
julia> x
(4, 5, 6)
julia> x[1]
4
```

SINGLE-ELEMENT TUPLE

```
x = (10,)    # not x = (10) (that's just x = 10)
```

```
julia> x
(10,)
julia> x[1]
10
```

TUPLES FOR ASSIGNMENTS

Tuples can be used to assign values to multiple variables at once. This operation is commonly referred to as **unpacking** or **destructuring**. It's implemented by placing both a tuple on the left-hand side of `=` and a collection on the right-hand side, where the latter may be either another tuple or a vector.

A TUPLE ON THE RIGHT-HAND SIDE

```
(x,y) = (4,5)
 x,y  =  4,5       #alternative notation
```

```
julia> x
4
julia> y
5
```

A VECTOR ON THE RIGHT-HAND SIDE

```
(x,y) = [4,5]
 x,y  = [4,5]      # alternative notation
```

```
julia> x
4
julia> y
5
```

The technique is commonly employed when a function returns multiple values, providing convenient syntax to unpack outputs into individual variables.